UNIT-2

PROBLEM SOLVING AND SEARCH

In which we see how an agent can look ahead to find a sequence of actions that will eventually achieve its goal.

Prepared by:
PRASHANTH KAMBLI,
ISE, RIT

Classified as Microsoft Confidential

Problem-solving agent :

- When the best action is not immediately clear, an agent plans ahead by evaluating a sequence of actions to reach a desired goal.
- Such an agent is known as a **Problem-Solving Agent**.
- The computational process it undertakes is called **search**.
- Problem-solving agents use **atomic representations** - that Search is, states of the world are considered as wholes, with no internal structure visible to the problem-solving algorithms.
- Agents that use **factored or structured** representations of states are called **planning agents**

- We will cover several search algorithms.
- In this chapter, we consider only the simplest environments: episodic, single agent, fully observable, deterministic, static, discrete, and known.
- We distinguish between **informed algorithms**, in which the agent can estimate how far it is from the goal, and **uninformed algorithms**, where no such estimate is available.

PROBLEM-SOLVING AGENTS

A student needs to reach the **exam hall before 9:00 AM**. There are several routes available: Bus, Metro, Cab. Each option differs in travel time, cost, and traffic conditions. The student must analyse the alternatives and select the best route to reach the goal on time.

This decision-making process illustrates a **Problem-Solving Agent.**

A Problem-Solving Agent searches for the best sequence of actions to reach a goal when the solution is not immediately obvious.

If the agent has no additional information—that is, if the environment is **unknown**—then the agent can do no better than to execute one of the actions at random.

THE AGENT CAN FOLLOW FOUR-PHASE PROBLEM-SOLVING PROCESS

1. Goal formulation

Scenario: A student needs to reach the exam hall before 9:00 AM using Bus, Metro, or Cab.

Goal Formation : Goal formation is the process of defining **what the agent wants to achieve** before deciding **how to achieve it**.

Goal: Reach the exam hall before 9:00 AM.

Components of Goal Formation

- **Current State:** Student is at home/hostel.
- **Goal State:** Student arrives at the exam hall before 9:00 AM.
- **Constraints:**
  - ➢ Limited time
  - ➢ Travel cost
  - ➢ Traffic conditions
- **Possible Actions:**
  - ✓ Take Bus
  - ✓ Take Metro
  - ✓ Take Cab

Goal formation answers "What should be achieved?"
while problem solving answers "How can it be achieved?".

2. Problem formulation

- The agent devises a description of the states and actions necessary to reach the goal
- Problem formulation is the process of defining the problem in terms of **states, actions, goal, and constraints** so that the agent can search for a solution

Example Scenario

A student needs to reach the **exam hall before 9:00 AM**.

HOW THE PROBLEM IS FORMULATED

1. **Initial State :** Student is at **Home/Hostel**.
2. **Goal State : s**tudent reaches the **Exam Hall before 9:00 AM**.
3. **Possible Actions**
    - Take a Bus
    - Take a Metro
    - Book a Cab
    - Walk between locations

```
Home
 ↓
Bus Stop / Metro Station / Cab
 ↓
Traveling
 ↓
Exam Hall
```

4. **State Space**

Possible states through which the student may pass:

5. Constraints

- Time available before 9:00 AM
- Travel cost
- Traffic conditions
- Route availability

WHY THIS IS PROBLEM FORMULATION

Before deciding which route to take, the student (agent) creates a **simplified model of the real world**:

- Considers only relevant information:
  - Time
  - Cost
  - Traffic
  - Transportation options
- Ignores irrelevant information:
  - Bus colour
  - Weather advertisements
  - Number of passengers
  - Student's clothing

This simplified representation is called an **abstract model**.

Problem Formulation

- The agent defines the problem using states, actions, goal, and constraints.
- **Initial State:** Student at home.
- **Goal State:** Reach exam hall before 9:00 AM.
- **Actions:** Bus, Metro, or Cab.
- **Constraints:** Time, cost, and traffic.
- The agent considers only relevant information and ignores unnecessary details.

Key Point:

Problem formulation converts a real-world situation into an abstract model consisting of states, actions, goals, and constraints that can be solved systematically.

3. Search

- Before taking any action in the real world, the agent simulates sequences of actions in its model, searching until it finds a sequence of actions that reaches the goal.
- Solution Such a sequence is called a solution.
- The agent might have to simulate multiple sequences that do not reach the goal, but eventually it will find a solution or it will find that no solution is possible.

SEARCH AND SOLUTION (STUDENT REACHING EXAM HALL EXAMPLE)

- After formulating the problem, the student (agent) must decide **which route to take**.
- Instead of immediately taking an action, the agent first **evaluates different possible routes in its mind (model)** and predicts their outcomes.

Search: is the process of exploring different sequences of actions to determine which one achieves the goal.

Example

Goal: Reach the exam hall before 9:00 AM.

Sequence 1

```
Home → Bus Stop → Bus → Exam Hall
```

- Estimated arrival: 9:15 AM ❌
- Goal not achieved

Sequence 2

```
Home → Metro Station → Metro → Exam Hall
```

- Estimated arrival: 8:45 AM ✅
- Goal achieved

Sequence 3

```
Home → Cab → Exam Hall
```

- Estimated arrival: 8:55 AM ✅
- Goal achieved
- Higher cost

The agent evaluates these possibilities before making a real-world decision.

SOLUTION

A **solution** is the sequence of actions that successfully reaches the goal.

For example:

```
Home → Metro Station → Metro → Exam Hall
```

is a solution because the student reaches the exam hall before 9:00 AM

Among multiple solutions, the agent may choose the **best solution** based on criteria such as:

- Shortest travel time
- Lowest cost
- Least traffic
- Highest reliability

Search

- The agent explores different action sequences in its internal model.
- It predicts the outcome of each option before acting.
- Some routes may fail to achieve the goal.
- The search continues until a goal-reaching path is found.

Solution

- A solution is a sequence of actions that reaches the goal.
- Example:
  - **Home → Metro Station → Metro → Exam Hall**
- The selected path enables the student to arrive before 9:00 AM.

Search is the process of exploring possible paths, while a solution is the path that successfully achieves the goal.

4. Execution

After the **search process** identifies the best solution, the agent must **carry out the chosen actions in the real world**.

This stage is called **Execution**.

Example

Suppose the search process determines that the best solution is:

```
Home → Metro Station → Metro → Exam Hall
```

During **Execution**, the student performs these actions one by one:

1. Leave home.
2. Walk to the metro station.
3. Board the metro.
4. Get off at the destination station.
5. Walk to the exam hall.
6. Arrive before 9:00 AM.

The student is no longer evaluating alternatives; instead, they are **following the selected solution path**.

Complete Flow

```
Goal Formation
        ↓
Problem Formulation
        ↓
Search
        ↓
Solution Found
        ↓
Execution
        ↓
Goal Achieved
```

Execution is the process of performing the actions of the selected solution to achieve the desired goal in the real world.

OPEN-LOOP VS CLOSED-LOOP SYSTEM
(STUDENT REACHING THE EXAM HALL EXAMPLE)

Goal: Reach the Exam Hall before 9:00 AM

Open-Loop System

- Plans the route once and follows it without checking the outcome.
- Example: Student chooses **Metro** and continues the journey regardless of delays.
- **No feedback** from the environment.
- Simple but cannot react to unexpected changes.

Closed-Loop System

- Continuously monitors the environment and adjusts actions if needed.
- Example: Student chooses **Metro**, notices a delay, and switches to a **Cab** to reach on time.
- **Uses feedback** (time, traffic, delays).
- More reliable and adaptive.

SEARCH PROBLEMS AND SOLUTIONS

1. State Space

A search problem can be defined formally as follows:

A set of possible states that the environment can be in.

We call this the state space.

All possible situations (states) the student can be in.

{Home, Bus Stop, Metro Station, In Bus, In Metro, In Cab, Near Exam Hall, Exam Hall}

State Space = All possible locations/states of the student

2. Initial State

Definition: The starting state of the agent.

Initial State = Home

3. Goal State

Definition: The desired state that the agent wants to achieve.

Reach Exam Hall before 9:00 AM

4. Actions

Definition: Possible actions available to the agent in a state.

ACTIONS(Home) = {Take Bus, Take Metro, Book Cab}

Possible actions:

- Walk to Bus Stop
- Board Bus
- Walk to Metro Station
- Board Metro
- Book Cab

5. Transition Model

Definition: Describes the result of performing an action.

RESULT(Home, Take Metro) = Metro Station

RESULT(Metro Station, Board Metro) = In Metro

RESULT(In Metro, Exit Metro) = Near Exam Hall

Transition Model tells us what happens after an action is executed.

6. Action Cost Function

- An **action cost function**, denoted by ACTION-COST(s,a,s`) when we are programming or c(s,a,s`) when we are doing math, that gives the numeric cost of applying action a in state s to reach state s`.
- A problem-solving agent should use a cost function that reflects its own performance measure
- An **Action Cost Function** assigns a numerical cost to performing an action and moving from one state to another.
- The agent uses these costs to choose the most efficient path to achieve the goal.

ACTION-COST(Home, Metro, Exam Hall) = 40
``

ACTION-COST(Home, Bus, Exam Hall) = 60

ACTION-COST(Home, Cab, Exam Hall) = 30

Goal: Reach the Exam Hall before 9:00 AM

Action	From State	To State	Cost
Take Bus	Home	Exam Hall	60 min
Take Metro	Home	Exam Hall	40 min
Take Cab	Home	Exam Hall	30 min

Agent's Decision

- If the performance measure is **minimum travel time** → Choose **Cab (30 min)**.
- If the performance measure is **minimum cost** → Choose **Bus**.
- If the performance measure is **balance of time and cost** → Choose **Metro**.

Key Point

✅ **Action Cost Function = Numeric measure of the cost of an action (time, money, distance, or energy).**

✅ **A problem-solving agent selects the path with the lowest overall cost according to its performance measure.**

SOME TERMINOLOGIES

- A sequence of actions forms a **path**
- A **solution** is a path from the initial state to a goal Path state.
- We assume that action costs are **additive**; that is, the total cost of a path is the sum of the individual action costs.
- An **optimal solution** has the lowest path cost among all solutions.
- The state space can be represented as a **graph** in which the vertices are states and the directed edges between them are **actions**.
- The map of Romania shown in Figure 3.1 is such a graph, where each road indicates two actions, one in each direction.

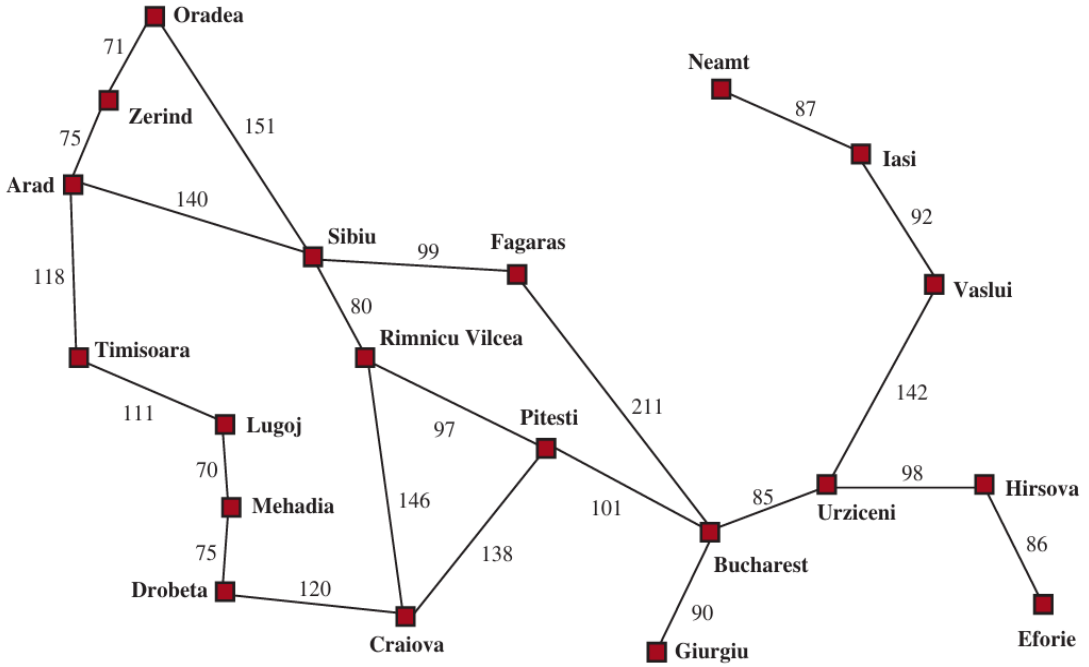


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

FORMULATING PROBLEMS

What is Problem Formulation?

- Problem formulation is the process of representing a real-world problem as an **abstract model.**
- The model includes only the information necessary to achieve the goal.
- It helps the agent focus on relevant details and ignore unnecessary ones.

Example: Route Planning

Goal: Reach the Exam Hall before 9:00 AM

Relevant Information

- Current location
- Available routes
- Travel time
- Cost
- Traffic conditions

Irrelevant Information

- Music being played
- Student's clothing
- Advertisements on the road
- Number of passengers

Key Point: Problem formulation converts a complex real-world situation into a simplified model that can be solved efficiently.

ABSTRACTION

What is Abstraction?

- **Abstraction** is the process of removing unnecessary details from a problem representation.
- It allows the agent to focus on important states and actions.

Example

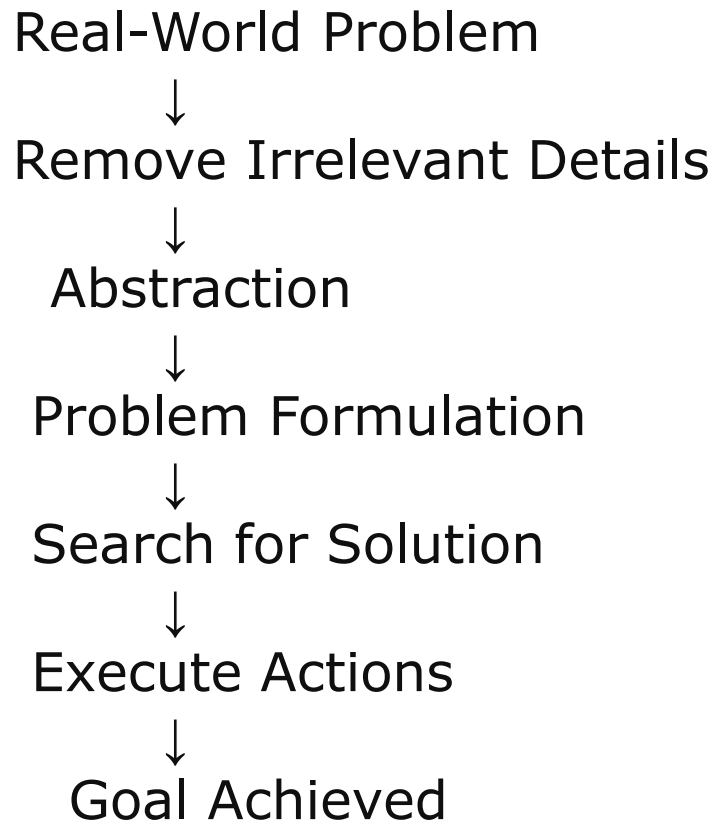
Real-World Problem

- Student + Weather + Friends +
- Traffic + Phone Notifications +
- Route Options + Travel Cost

Abstract Problem

- Home → Metro → Exam Hall

Key Point:
Abstraction simplifies decision-making by keeping only goal-relevant information.



STANDARDIZED PROBLEMS VS REAL-WORLD PROBLEMS

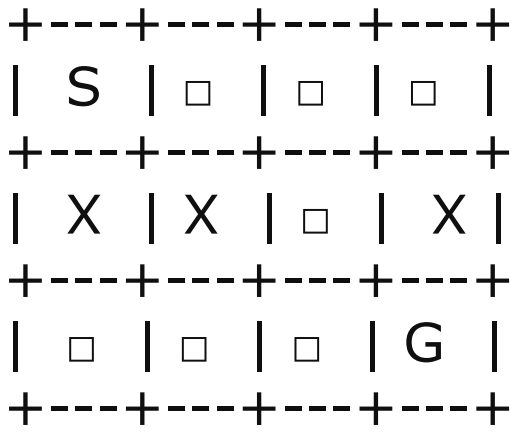
Standardized Problems

- Well-defined and commonly used for teaching and evaluating AI algorithms.
- Have a concise and exact problem description.
- Serve as benchmarks for comparing search and problem-solving techniques.
- Same problem setup is used by all researchers.

Examples:

- Romania Route-Finding Problem
- 8-Puzzle Problem
- Vacuum Cleaner World
- Grid World

EXAMPLE : SOLVABLE GRID WORLD



Legend:

- **S** = Start State
- **G** = Goal State
- **X** = Obstacle
- □ = Free Cell

(1,1) S
→
(1,2)
→
(1,3)
↓
(2,3)
↓
(3,3)
→
(3,4) G

Visualized Path

S → → ↓
X X ↓ X
□ □ → G

Search Problem Components

- **Initial State:** S
- **Goal State:** G
- **Actions:** Up, Down, Left, Right
- **Obstacles:** X
- **Path Cost:** 1 per move
- **Solution Path:** S → (1,2) → (1,3) → (2,3) → (3,3) → G
- **Total Cost:** 5 moves

Path to Goal : The agent can take the following path S → → ↓ ↓ →

In a Grid World, the agent searches through available cells while avoiding obstacles to find a valid path from the Start State (S) to the Goal State (G).

8-PUZZLE PROBLEM

What is the 8-Puzzle Problem?

The **8-Puzzle** is a classic AI search problem consisting of:

- A **3 × 3 grid**
- **8 numbered tiles (1–8)**
- **1 blank space (□)**

The objective is to rearrange the tiles from an **initial state** to a **goal state** by sliding one tile at a time into the blank space.

EXAMPLE

Initial State

1	2	3
4	□	6
7	5	8

Goal State

1	2	3
4	5	6
7	8	□

Actions

Move the blank space:

- Up (↑)
- Down (↓)
- Left (←)
- Right (→)

Solution:

Move 5 Up → Move 8 Left

Cost Function

- Cost = 1 per move
- Goal: Reach the goal state with minimum moves

Key Point:

The 8-Puzzle is a standardized search problem where an agent searches for the sequence of moves that transforms the initial state into the goal state with minimum cost.

Real-World Problems

- Represent practical situations encountered in everyday life.
- Problem formulation varies depending on the environment and requirements.
- More complex due to uncertainty and changing conditions.
- Different agents may perceive the same problem differently.

Examples:

- Robot Navigation
- GPS Route Planning
- Autonomous Driving
- Medical Diagnosis
- Delivery Drone Routing

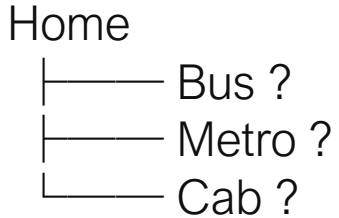
UNINFORMED SEARCH STRATEGIES

Definition

- **Uninformed Search (Blind Search)** explores the search space **without any additional** knowledge about how close a state is to the goal.
- The agent knows only:
  - ✓ Initial State
  - ✓ Available Actions
  - ✓ Goal Test
  - ✓ Path Cost (if applicable)
- It does **not** know which path is better or closer to the goal.

Uninformed Search (Blind Search)

- The student has **no information** about travel times, traffic, or route efficiency.
- All options (Bus, Metro, Cab) appear equally good.
- The student explores alternatives without knowing which one is closer to the goal.



Student thinks:

"I don't know which option will get me to the exam hall faster. Let me try/explore the possibilities."

Characteristics

- No guidance
- Systematic exploration
- May examine unnecessary routes
- More time and effort required

COMMON UNINFORMED SEARCH ALGORITHMS

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Depth-Limited Search (DLS)
- Iterative Deepening Search (IDS)
- Bidirectional Search

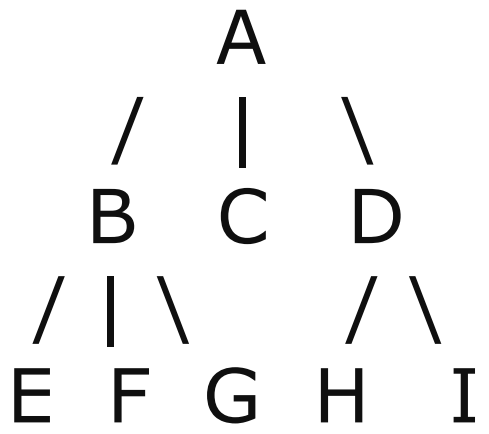
BREADTH-FIRST SEARCH

- **Breadth-First Search (BFS)** explores the search tree **level by level**.
- It first expands the **root node**, then all its immediate children, then their children, and so on.
- BFS is suitable when **all actions have the same cost**.
- It always finds the **shallowest goal node** (minimum number of steps).
- Uses a **FIFO Queue (First In, First Out)**.

Key Idea is :

Level 0 → Level 1 → Level 2 → Level 3

Explore all neighboring nodes before moving deeper.



FIFO QUEUE CONCEPT

Queue Principle

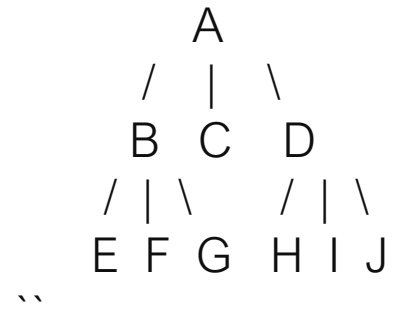
FIFO = First In First Out

Example

- Queue = [A, B]
- Remove A
- Queue = [B]
- Remove B
- Queue = [ ]

BFS uses a queue to maintain the order of node expansion.

BFS TRAVERSAL EXAMPLE



Sequence

A → B → C → D → E → F → G → H → I → J

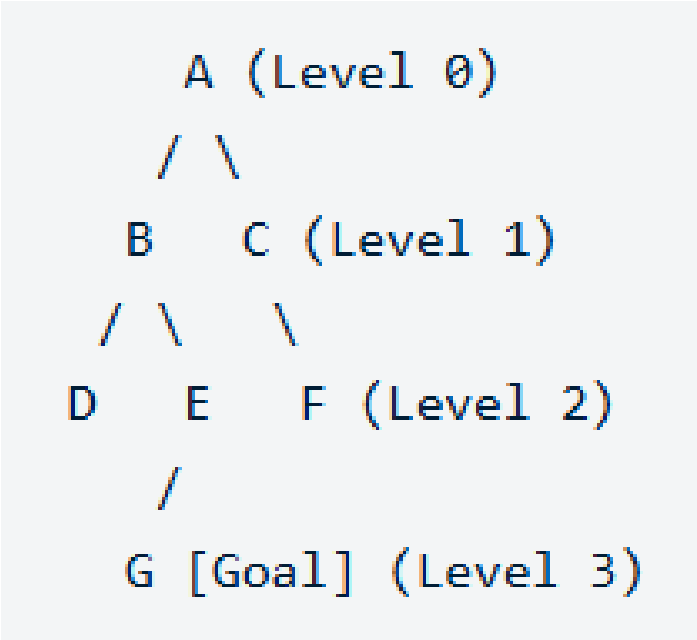
BFS Traversal Order

A
↓
B, C, D
↓
E, F, G, H, I, J

BFS WORKING STEP-BY-STEP

Step-by-Step Example

Consider the following search tree, where **A** is the start/root node, and **G** is our target Goal Node.



To prevent infinite loops, we keep track of a **Queue** (nodes waiting to be explored) and a **Visited list** (nodes already processed).

Step	Current Node	Queue State (Front → Rear)	Visited List / Order of Traversal	Action / Notes
0	—	`[A]`	`[]`	Initialize queue with root node **A**.
1	**A**	`[B, C]`	`[A]`	Remove **A**. It is not the goal. Push its children **B** and **C** to the queue.
2	**B**	`[C, D, E]`	`[A, B]`	Remove **B**. Push its children **D** and **E**.
3	**C**	`[D, E, F]`	`[A, B, C]`	Remove **C**. Push its child **F**. *(Level 1 is now fully explored).*
4	**D**	`[E, F]`	`[A, B, C, D]`	Remove **D**. It has no children.
5	**E**	`[F, G]`	`[A, B, C, D, E]`	Remove **E**. Push its child **G**.
6	**F**	`[G]`	`[A, B, C, D, E, F]`	Remove **F**. It has no children.
7	**G**	`[]`	`[A, B, C, D, E, F, G]`	Remove **G**. **Goal found!** Stop search.

Final Traversal Order: A → B → C → D → E → F → G

If **E** is the goal node, the final path found by BFS is **A → B → E**.

Even though BFS explores nodes level-by-level (visiting A, B, C, D, and then E), it keeps track of how it reached each node (parent pointers). When the algorithm hits the goal node, it traces those pointers back to the start to construct the final path:

1. **E** was reached from **B**
2. **B** was reached from **A**

Key Difference: Traversal Order vs. Final Path

- **Traversal Order (Nodes expanded):** A → B → C → D → E (This is the order the algorithm *searched* through the tree).
- **Final Path (The solution): A → B → E** (This is the actual route from start to finish).

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```

Figure 3.9 Breadth-first search and uniform-cost search algorithms.

BFS CHARACTERISTICS

Advantages

- ✅ Complete
- ✅ Finds shortest path when all step costs are equal
- ✅ Systematic search strategy

Disadvantages

- ❌ High memory requirement
- ❌ Large search spaces become expensive
- ❌ Explores many unnecessary nodes

KEY CHARACTERISTICS

- **Level-by-Level Exploration:** It visits all nodes at the current depth (shallowest first) before moving down to the next level.
- **Data Structure:** It relies on a **First-In, First-Out (FIFO) queue** to track which nodes to expand next.
- **Completeness & Optimality:** BFS is **complete** (it will always find a solution if one exists) and **optimal** for unweighted graphs (it guarantees finding the shortest path with the fewest steps).
- **Complexity:** Both time and space complexity are $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal node.

DEPTH-FIRST SEARCH (DFS)

- **Uninformed Depth-First Search (DFS)** is a blind search algorithm that explores a tree or graph by going as deep as possible down each branch before turning back.
- It is called "uninformed" (or blind) because it uses no extra information or clues (heuristics) about how close it is to the goal.
- It relies solely on the basic structure of the problem.
- It uses a **Stack** data structure that follows the **Last-In, First-Out (LIFO)**

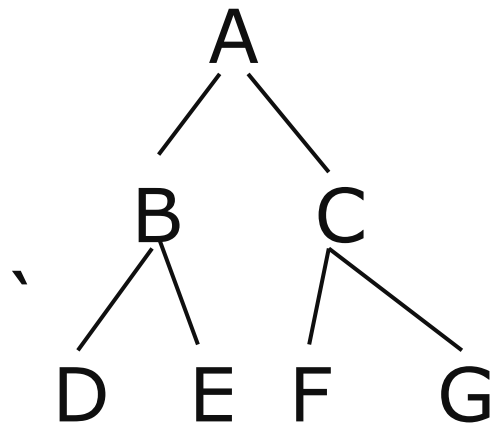
HOW DFS WORKS?

1. **Start** at the root node and add it to the stack.
2. **Mark** the current node as visited.
3. **Pick** one unvisited neighbouring node, go to it, and push it onto the stack.
4. **Repeat** this deep dive until you hit a dead end (a node with no unvisited neighbours).
5. **Backtrack** to the most recent node that still has unvisited paths, and explore those paths.
6. **Stop** when you find the goal or visit every reachable node.

STEP-BY-STEP EXAMPLE

Imagine a simple tree with nodes:

- **Root (Level 0):** A
- **Level 1:** B, C
- **Level 2:** D, E (children of B), and F, G (children of C)



THE TRAVERSAL STEPS:

1. **Visit A**: Start at the root node A. Push A to stack. Mark as visited.
2. **Go to B**: From A, the first child is B. Push B to stack. Mark B as visited.
3. **Go to D**: From B, the first child is D. Push D to stack. Mark D as visited.
4. **Dead end at D**: Node D has no children. Pop D from the stack. Backtrack to B.
5. **Go to E**: From B, the next unvisited child is E. Push E to stack. Mark E as visited.
6. **Dead end at E**: Node E has no children (or its children are visited). Pop E, pop B. Backtrack all the way back to A.
7. **Go to C**: From A, the next unvisited child is C. Push C to stack. Mark C as visited.
8. **Go to F**: From C, the first child is F. Push F to stack. Mark F as visited.
9. **Dead end at F**: Node F has no children. Pop F. Backtrack to C.
10. **Go to G**: From C, the next child is G. Push G to stack. Mark G as visited

Final Visit Order: A → B → D → E → C → F → G

KEY PROPERTIES OF DFS

- **Data Structure:** Stack (LIFO)
- **Space Complexity:** O(m), where m is the maximum length of a path. It is memory-efficient because it only stores nodes along the active path.
- **Time Complexity:** $O(bm)$, where b is the branching factor (number of children per node).
- **Completeness:** Not complete on infinite graphs or graphs with cycles unless you track visited nodes to avoid infinite loops.
- **Optimality:** Not optimal; it does not guarantee finding the shortest path to a goal.

UNIFORM COST SEARCH (UCS)

- **Uniform Cost Search (UCS)** is an **uninformed search algorithm**
- It finds the optimal path (the path with the absolute lowest cumulative cost) from a starting node to a destination node in a weighted graph.
- Unlike Breadth-First Search (BFS), which explores nodes layer-by-layer based on depth, UCS evaluates nodes based on their **cumulative path cost** from the root.
- It is structurally equivalent to a goal-oriented version of Dijkstra's algorithm.

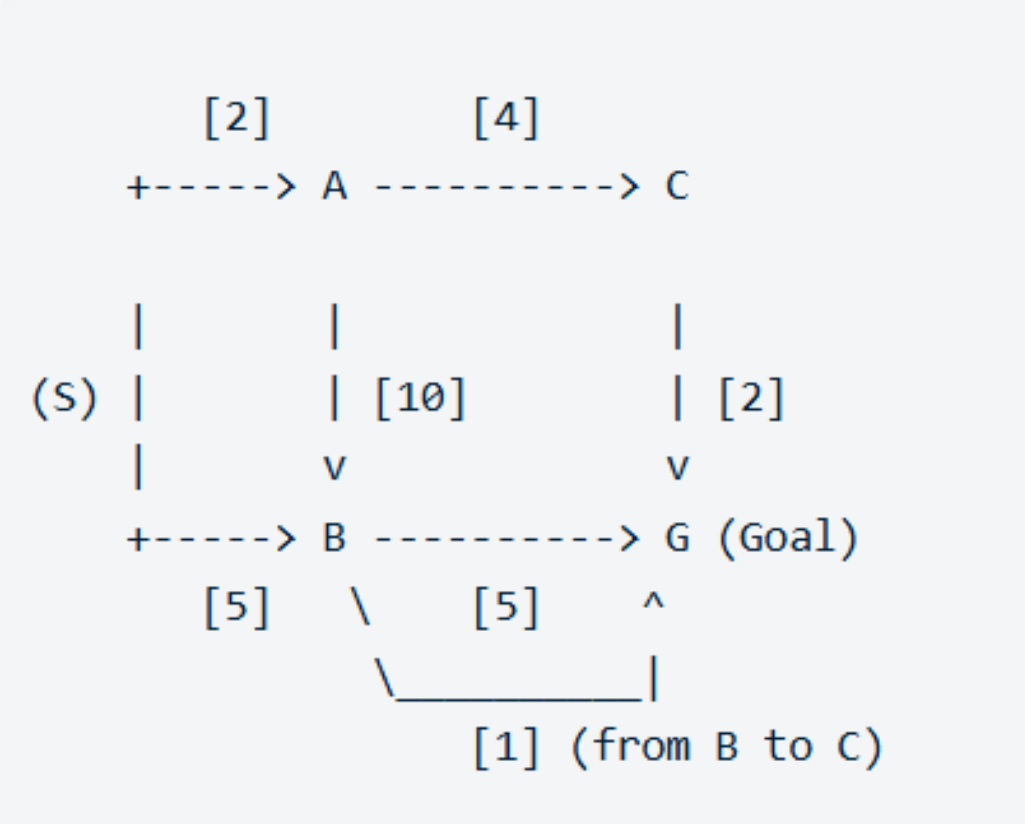
CORE MECHANICS & RULES

- UCS relies on a **Priority Queue** (often referred to as the *Frontier* or *Open List*) to decide which node to explore next.

1. **Cumulative Cost (g(n)):** The priority of any node n is the total cost accumulated from the start node to n. The node with the **lowest** g(n) is always placed at the front of the queue.
2. **Goal Check Constraint:** The algorithm **only** checks if a node is the goal when that node is *popped (expanded)* from the priority queue, not when it is first generated. This prevents it from prematurely settling on a sub-optimal path.
3. **Optimality:** UCS is guaranteed to find the cheapest path provided that all step costs are non-negative

STEP-BY-STEP EXAMPLE

- Consider the directed, weighted graph below. We want to find the lowest-cost path from the start node **S** to the goal node **G**.



The Graph Layout:

- S connects to: A (cost: 2), B (cost: 5)
- A connects to: C (cost: 4), G (cost: 10)
- B connects to: C (cost: 1), G (cost: 5)
- C connects to: G (cost: 2)

EXECUTION TRACE

We track two lists:

- **Priority Queue (Frontier):** Elements kept in order of [Node, Cumulative Cost].
- **Explored (Closed Set):** Nodes that have already been expanded.

Step	Node Popped	Neighbors Explored / Path Cost Updated	Priority Queue Status (Sorted)	Explored Set
0	—	*Initialize with Start Node*	`[(S, 0)]`	`{}`
1	**S**	A via S: 0 + 2 = 2 B via S: 0 + 5 = 5	`[(A, 2), (B, 5)]`	`{S}`
2	**A**	C via A: 2 + 4 = 6 G via A: 2 + 10 = 12	`[(B, 5), (C, 6), (G, 12)]`	`{S, A}`
3	**B**	C via B: 5 + 1 = 6 *(No cheaper, ignore)* G via B: 5 + 5 = 10 *(Cheaper than 12! Update G)*	`[(C, 6), (G, 10)]`	`{S, A, B}`
4	**C**	G via C: 6 + 2 = 8 *(Cheaper than 10! Update G)*	`[(G, 8)]`	`{S, A, B, C}`
5	**G**	**Goal Found!** Algorithm stops immediately.	`[]`	`{S, A, B, C, G}`

Final Output:

- **Optimal Path:** S → A → C → G
- **Minimum Total Cost: 8**

(Note: Even though the path S → B → G takes fewer jumps, UCS uncovers S → A → C → G because its total mathematical cost is lower.)

ALGORITHM PROPERTIES

- **Completeness:** Complete if the branching factor is finite and every step cost is greater than a tiny positive value ε (to prevent infinite loops on zero-cost paths)
- **Optimality: Yes**, always optimal if edge costs are non-negative.
- **Time & Space Complexity:**, $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where C^* is the cost of the optimal solution, b is the branching factor, and ε is the minimum step cost. Space and time requirements are heavily dependent on path cost rather than depth.

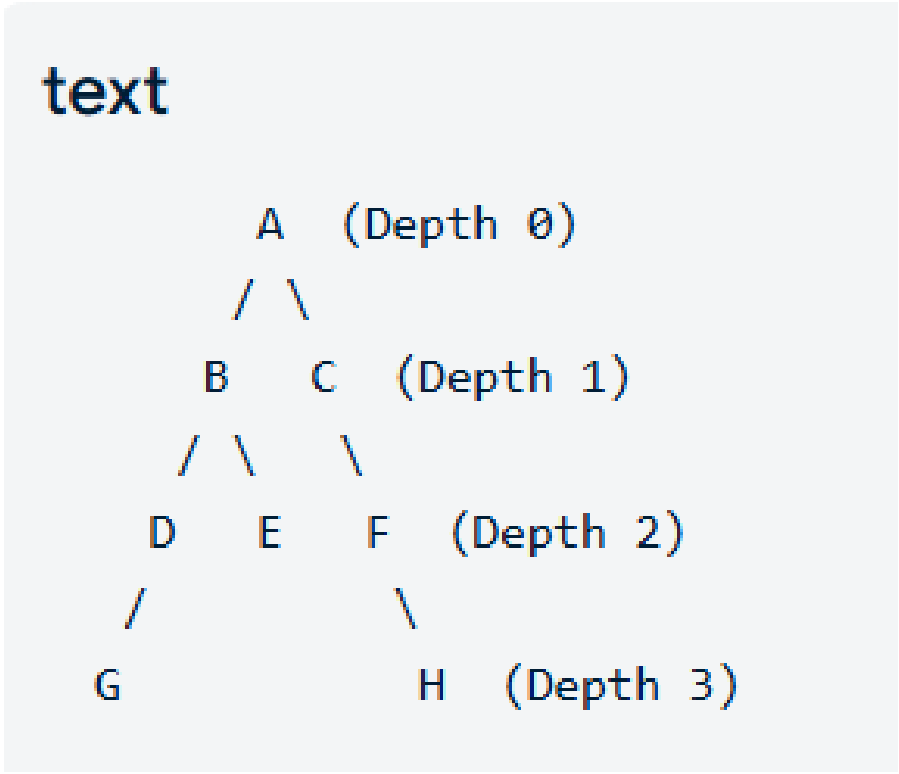
DEPTH-LIMITED SEARCH (DLS)

- **Depth-Limited Search (DLS)** is an uninformed (blind) search algorithm that explores a search space using a depth-first approach but caps the exploration at a predefined depth limit (L).
- It directly addresses the primary flaw of standard Depth-First Search (DFS)—falling into infinite loops or getting lost down infinitely deep branches.
- Once the algorithm reaches the depth limit L, it treats those nodes as if they have no children (successors) and backtracks to explore alternative paths.

STEP-BY-STEP EXAMPLE

- Consider the following search tree where **Node A** is the root (initial state) and **Node G** is the target Goal.

text



SCENARIO 1: DEPTH LIMIT L = 2 (WE WANT TO FIND THE GOAL **G**, BUT WE SET OUR CUTOFF LIMIT TO **2**)

1. **Visit A** (Depth 0): Not the goal. Move to its first child.
2. **Visit B** (Depth 1): Not the goal. Move to its first child.
3. **Visit D** (Depth 2): Not the goal. Because D is at the depth limit (L=2), the algorithm **stops expanding** its children (it ignores Node G at Depth 3).
4. **Backtrack to B**, then move to its next child, **E** (Depth 2): Not the goal. It has reached the limit, so it backtracks.
5. **Backtrack to A**, then move to its next child, **C** (Depth 1): Not the goal. Move to its child.
6. **Visit F** (Depth 2): Not the goal. Reaches the limit, so it stops.
7. **Terminates with Cutoff Failure:** The search ends because all paths up to level 2 have been exhausted. The goal **G** is missed because it sits at Depth 3

SCENARIO 2: DEPTH LIMIT L = 3 (WE INCREASE OUR CUTOFF LIMIT TO **3** TO ALLOW DEEPER EXPLORATION)

- **Visit A** → **Visit B** → **Visit D**.
- **Visit G** (Depth 3): The algorithm checks Node G. Since it matches the target condition, the search **succeeds** and returns the path: A -> B -> D -> G

CORE CHARACTERISTICS

- **Time Complexity:** $O(b^L)$, where b is the branching factor and L is the depth limit.
- **Space Complexity:** $O(b \times L)$, meaning it retains the linear memory efficiency of DFS.
- **Completeness:** Incomplete if the goal node resides at a depth greater than L (d > L).
- **Optimality:** Non-optimal, as it may find a deeper goal node before exploring a shallower path if the shallower path lies along a different branch.

ITERATIVE DEEPENING SEARCH (IDS)

- **Iterative Deepening Search (IDS)**, also known as Iterative Deepening Depth-First Search (IDDFS), is an uninformed search strategy that combines the space efficiency of Depth-First Search (DFS) with the completeness and optimality of Breadth-First Search (BFS).
- **Depth-Limited Search (DLS)**, starting with a depth limit of 0. If the goal is not found, it increments the depth limit by 1, restarts the search from the root node, and repeats the process until the goal is discovered.

STEP-BY-STEP EXAMPLE

- Consider the following tree structure where **Node A** is the Start state and **Node G** is the Goal state.

text

```
        A (Depth 0)
       / \
      B   C (Depth 1)
     / \   \
    D   E   G [GOAL] (Depth 2)
```

The algorithm will explore the tree across successive iterations:

1. **Iteration 0 (Depth Limit = 0)**
   - The search is restricted strictly to Level 0.
   - It visits the root node: **A**.
   - **A** is checked. It is not the goal.
   - *Iteration 0 terminates*

2. **Iteration 1 (Depth Limit = 1)**
   - The search restarts from the root and can explore up to Level 1.
   - It performs a standard DFS up to this limit:
     1. Visit **A**
     2. Move down to **B** (checks if it's the goal; it is not)
     3. Backtrack to A, then move to **C** (checks if it's the goal; it is not)
   - *Iteration 1 terminates without finding the goal.*

- **3. Iteration 2 (Depth Limit = 2)**
- The search restarts from the root again, now allowed to explore down to Level 2.
- It executes a DFS up to Level 2:
    1. Visit **A**
    2. Move to **B**
    3. Move to **D** (Level 2 limit reached; checks D; not the goal; backtracks)
    4. Move to **E** (Level 2 limit reached; checks E; not the goal; backtracks)
    5. Backtrack all the way to A, then move to **C**
    6. Move to **G** (Level 2 limit reached)
- **G is the Goal state!** The search stops successfully and returns the path: **A** → **C** → **G**.

WHY USE ITERATIVE DEEPENING?

- **Memory Efficient:** Like DFS, it only needs to store the current path it is exploring. Its space complexity is O(bd), where b is the branching factor and d is the depth of the shallowest goal. (Compare this to BFS, which stores all nodes in memory and easily runs out of RAM).
- **Complete and Optimal:** Like BFS, it is guaranteed to find a solution if one exists, and it will find the shortest path first in an unweighted graph.
- **Handles Infinite Paths:** Unlike pure DFS, it won't get trapped going down an infinite branch or cycle because the depth limit forces it to backtrack.

INFORMED SEARCH (HEURISTIC SEARCH)

- **Informed Search** uses additional knowledge about the problem to guide the search toward the goal.
- This extra knowledge is called a **heuristic.**
- The heuristic estimates how close a state is to the goal.
- Helps find solutions more efficiently than uninformed search methods.
- The hints come in the form of a heuristic function, denoted ***h(n)= estimated cost of the cheapest path from the state at node n to a goal state.***

Current State
↓
Heuristic Function h(n)
↓
Estimates Distance
↓
Choose Best Node
↓
Reach Goal Faster

Advantages

- Explores fewer nodes
- Faster search
- Reduces computational cost
- Suitable for large search spaces

HEURISTIC SEARCH

What is Heuristic Search?

Definition

- **Heuristic Search** is an informed search strategy.
- Uses additional knowledge called a **heuristic** to guide the search.
- Tries to reach the goal faster by selecting the most promising node.
- Also known as **Intelligent Search**.

Use Experience/Knowledge
↓
Choose Best Path
↓
Reach Goal Faster

BEST EXAMPLE FOR HEURISTIC SEARCH: **GOOGLE MAPS NAVIGATION**

Scenario : You are traveling from **Home** to **College**.

Available Routes:

Home
├── Route A (12 km)
├── Route B (8 km)
└── Route C (15 km)

Without Heuristic (Uninformed Search)

The agent explores:

Route A ?
Route B ?
Route C ?

It does not know which route is better.

With Heuristic Search

Google Maps uses:

- Estimated distance
- Travel time
- Traffic conditions
- Road closures

The heuristic value is:

h(Route A) = 25
h(Route B) = 15
h(Route C) = 35

Route A = 25 min
Route B = 15 min
Route C = 35 min

Since Route B has the smallest estimate:

Choose Route B

Goal reached faster.

Heuristic Search uses estimated information (distance, time, cost, etc.) to choose the most promising path and reach the goal more efficiently.

HEURISTIC FUNCTION

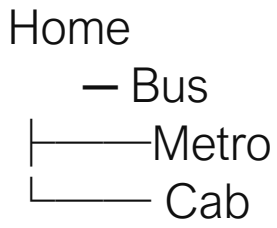
- A **Heuristic Function**, denoted as **h(n)**, estimates the cost or distance from the current node **n** to the goal state.

h(n) = Estimated cost from Node n to Goal

- It helps the agent decide which path appears more promising.
- Lower heuristic value ⇒ Closer to the goal.
- Used in informed search algorithms such as **Greedy Best-First Search** and **A***.

EXAMPLE

Goal: Reach the Exam Hall



Estimated Travel Time

Route	h(n)
Bus	60 min
Metro	40 min
Cab	30 min

Heuristic Values

h(Bus) = 60
h(Metro) = 40
h(Cab) = 30

Decision

Choose Cab
because h(Cab) is minimum

The agent chooses the route that appears closest to the goal.

A Heuristic Function h(n) estimates how close a state is to the goal and guides the search toward the most promising path.

COMPARISION

Uninformed Search	**Informed Search**
No heuristic	Uses heuristic
Blind exploration	Guided exploration
More nodes explored	Fewer nodes explored
Usually slower	Usually faster

GREEDY BEST-FIRST SEARCH (GBFS)

- **Greedy Best-First Search (GBFS)** is an **Informed Search Algorithm.**
- It always expands the node that appears **closest to the goal.**
- Uses only the **heuristic function h(n).**
- Chooses the most promising node at every step.

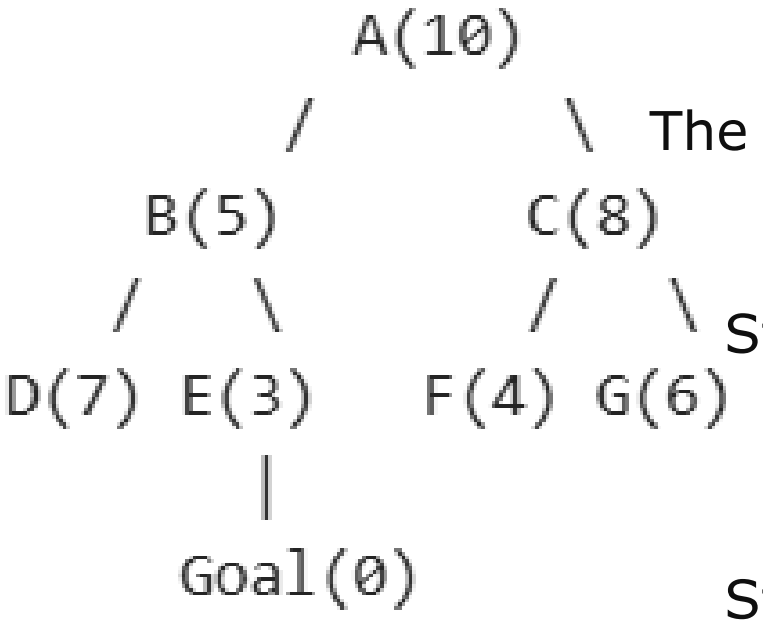
Evaluation Function

$$f(n) = h(n)$$

Where: h(n) = Estimated cost from n to Goal

Choose the node with the smallest h(n)

EXAMPLE



The number inside brackets is the **heuristic value h(n).**

Rule : Choose the node with the smallest h(n)

Step 1: Start at Root

Current Node: A(10)

Children Node: B(5) , C(8)

Step 2: Select Minimum h(n)

Choose: B(5) because 5 < 8

Children Node: D(7) , E(3)

Step 3: Select Minimum h(n)

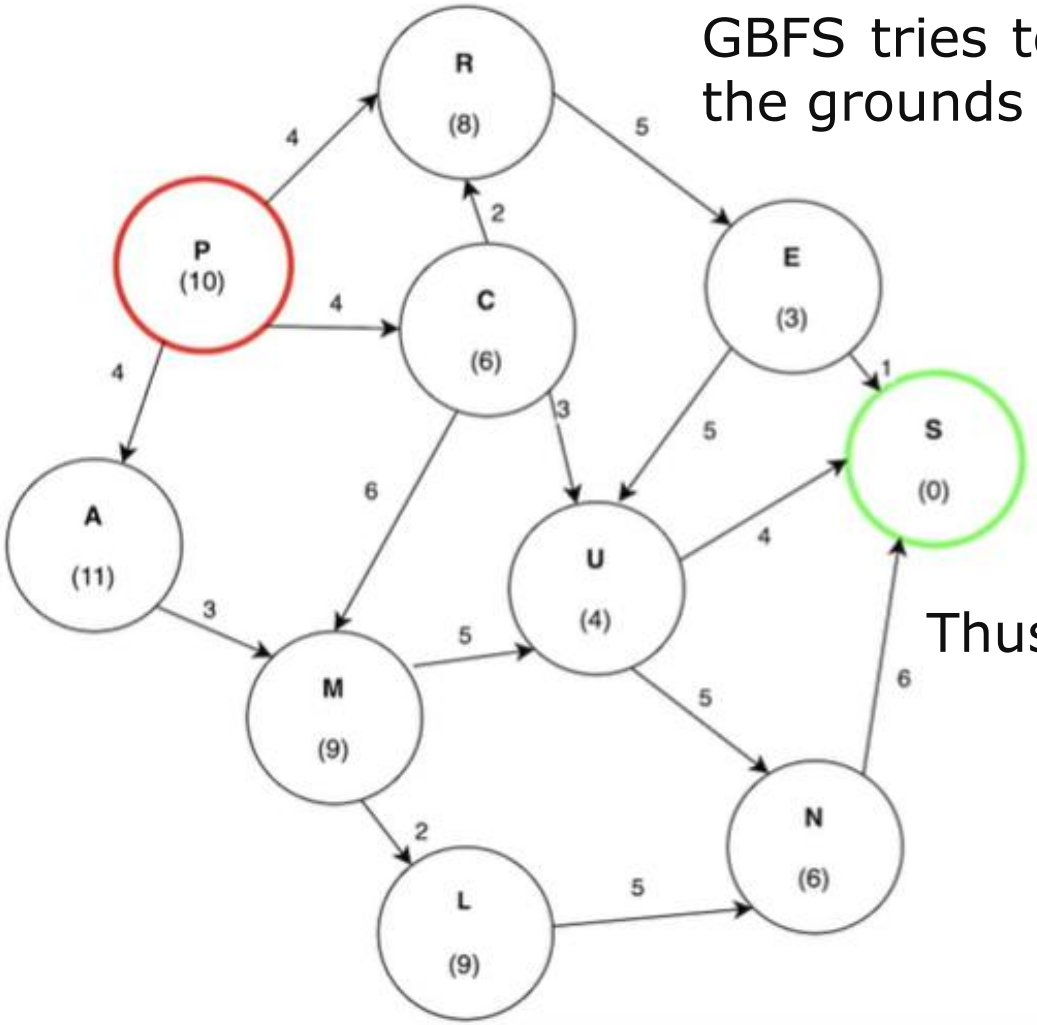
Choose: E(3) because 3 < 7

Children Node: Goal (0)

Path : A(10) → B(5) → E(3) → Goal (0)

Step	Open List	Selected Node
1	A(10)	A
2	B(5), C(8)	B
3	C(8), D(7), E(3)	E
4	C(8), D(7), Goal(0)	Goal

EXAMPLE 2



GBFS tries to expand the node that is **closest to the goal**, on the grounds that it is likely to **lead to a solution quickly**.

Thus, it evaluates nodes by using the heuristic function

f(n) = h(n)

h(n) = estimated cost from the state at node n to a goal state

Step	Open List	Selected Node
1	**P(10)**	**P**
2	**A(11), C(6), R(8)**	**C**
3	**M(9), U(4), R(8)**	**U**
4	**N(6), S(0) Goal**	**S**

Path : P(10) → C(6) → U(4) → S Goal (0)

Path Cost = 11

Note: This algorithm is not optimal because as it concentrates only on heuristic value but not cost

ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Fast search	Not Optimal
Explores fewer nodes	May choose a wrong path
Easy implementation	Depends heavily on heuristic quality
Suitable for large search spaces	Can get trapped in local minima

A* SEARCH

- **A*** is the most popular **Informed Search Algorithm**.
- It combines:
  - Actual cost from Start → Current Node
  - Estimated cost from Current Node → Goal
- Finds the **optimal (shortest) path** if the heuristic is admissible.

Evaluation Function : f(n) = g(n) + h(n)

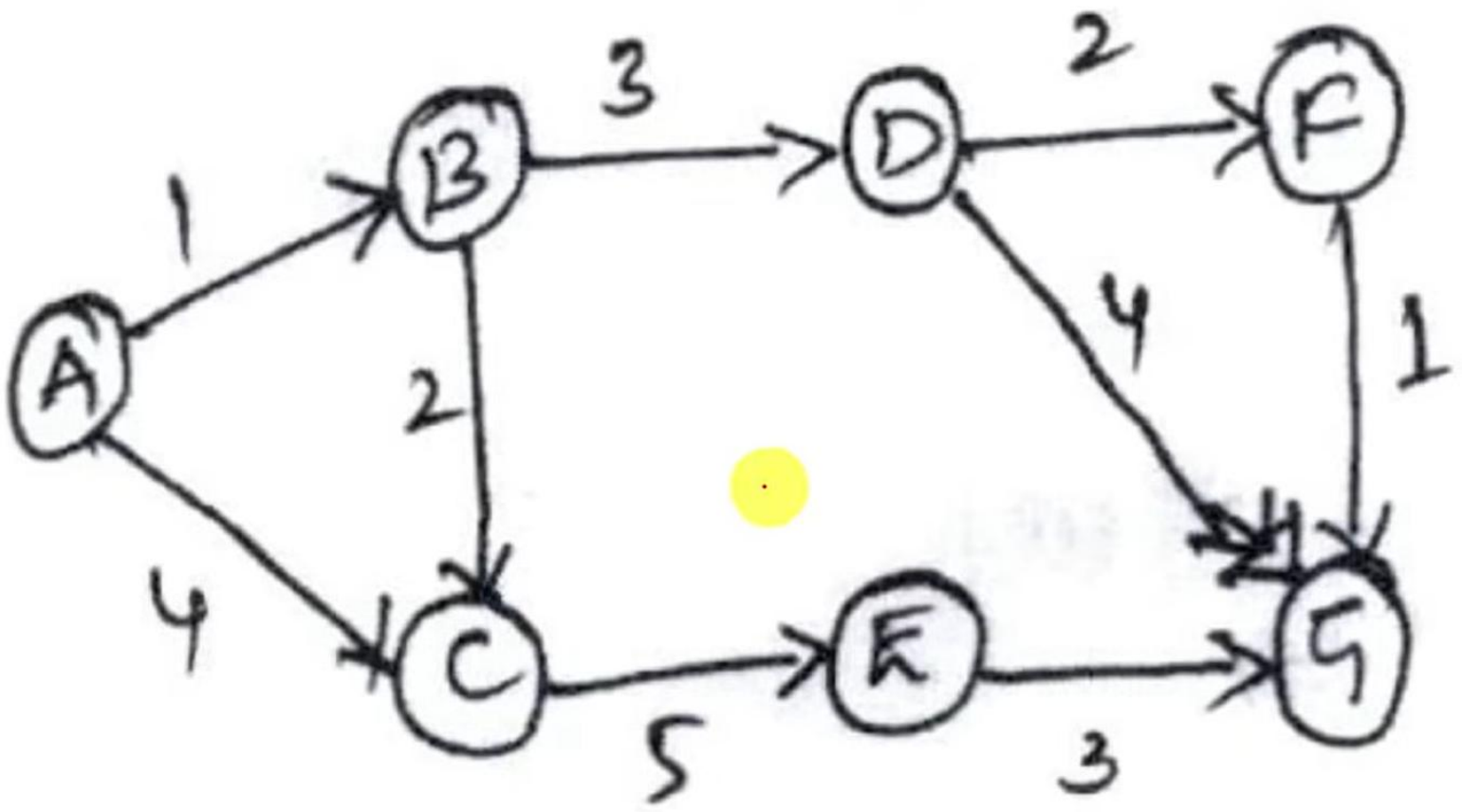
Where:

g(n) = Actual cost from Start to n

h(n) = Estimated cost from n to Goal

f(n) = Total estimated cost

EXAMPLE

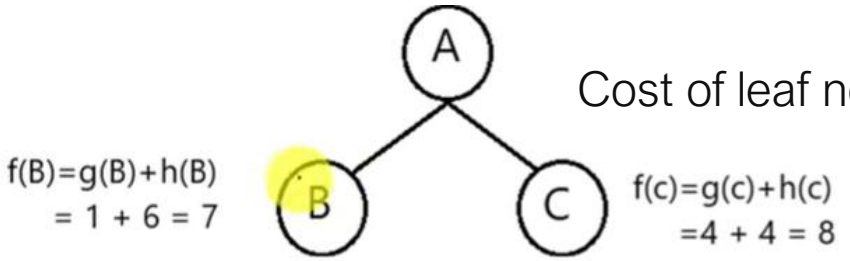


A	5
B	6
C	4
D	3
E	3
F	1
G	0

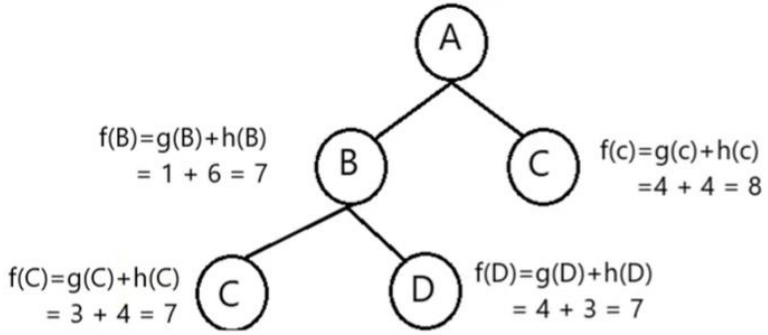


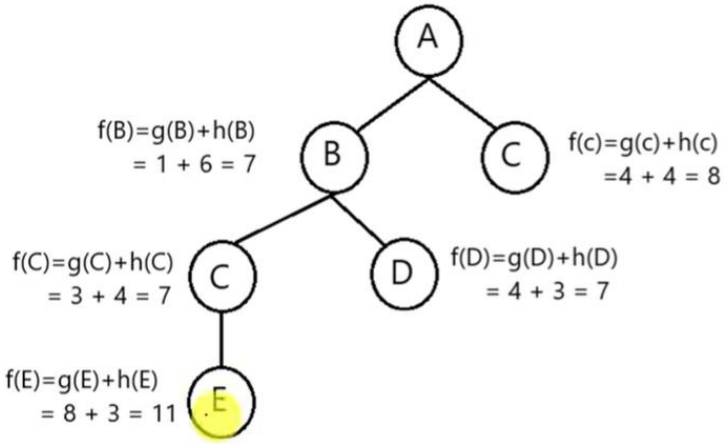
Start with Initial Stage A

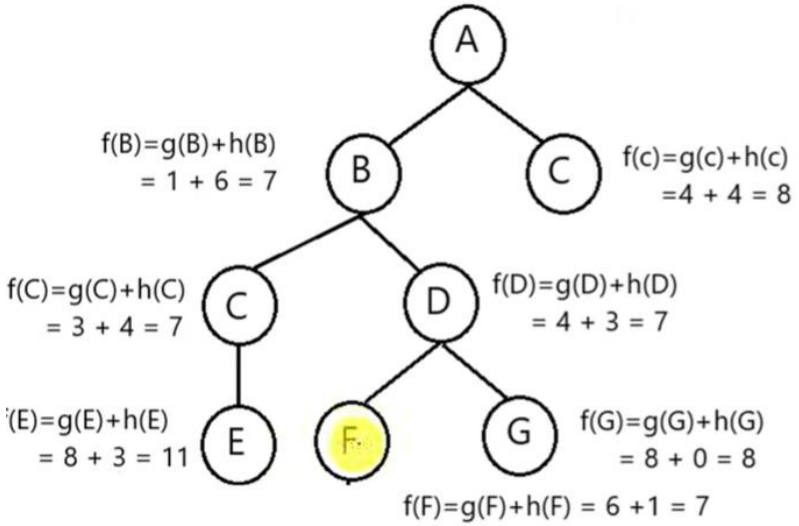
$$f(n) = g(n) + h(n)$$

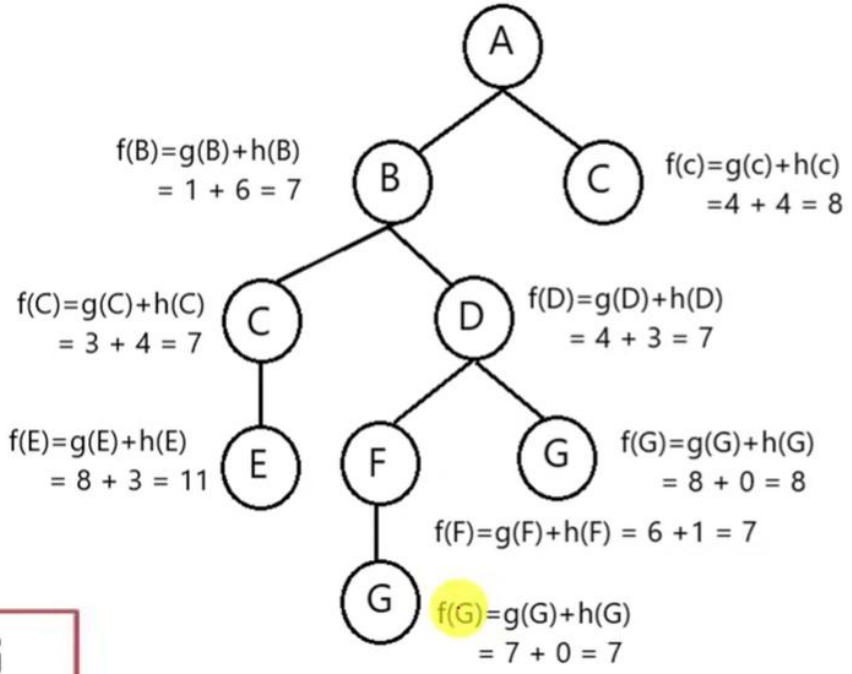


Cost of leaf nodes









Path: A→B→D→F→G
Path Cost: 7

ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Complete	Requires a good heuristic
Optimal	Higher memory usage
Efficient path finding	Performance depends on heuristic quality
Used in real-world applications • Google Maps • GPS Navigation • Robotics • Games • Autonomous Vehicles	

GREEDY BEST-FIRST SEARCH VS A*

Greedy Best-First Search	A* Search
Uses only h(n)	Uses g(n) + h(n)
Faster	Slightly slower
Not Optimal	Optimal
Less Memory	More Memory
Goal-Oriented	Cost + Goal-Oriented

ADMISSIBLE HEURISTICS

- A heuristic **h(n)** is **admissible** if it **never overestimates** the actual minimum cost to reach the goal.

Mathematically: $h(n) \leq h^*(n)$

Where:

h(n) = Heuristic estimate

h*(n) = Actual minimum cost to reach*the goal

Simple Meaning:

Underestimate is allowed
Exact estimate is allowed
Overestimate is NOT allowed

EXAMPLE

- Suppose the actual shortest distance to the college is:
- Actual distance = 10 km

Heuristic	Admissible?
6 km	Yes
8 km	Yes
10 km	Yes
12 km	No

- Heuristic ≤ Actual Cost

CONSISTENT HEURISTICS (MONOTONIC HEURISTICS)

- A heuristic is **consistent** if, for every node $n$and its successor n',
- h(n) ≤ c(n,n') + h(n’)

  Where:

  h(n) = Heuristic value of current node

  h(n') = Heuristic value of successor node

  c(n,n') = Cost to move from n to n'

The estimated distance to the goal should never decrease by more than the actual cost of moving to the next node.

EXAMPLE

College Route

Home ----5 km---- Metro ----3 km---- College

h(Home) = 8 km

h(Metro) = 3 km

$h(Home) \leq Cost(Home,Metro) + h(Metro)$

$8 \leq 5 + 3$

$8 \leq 8$

Consistent

INCONSISTENT HEURISTIC EXAMPLE

A(h=10)
|
cost=2
|
B(h=3)

Check consistency:

$10 \leq 2 + 3$
$10 \leq 5$

Not Consistent

WHY IS CONSISTENCY IMPORTANT IN A*?

- For A*:

$$f(n) = g(n) + h(n)$$

If the heuristic is consistent:

- ✅ A* never needs to revisit a node
- ✅ f(n) values never decrease along a path
- ✅ Search becomes more efficient
- ✅ Optimal solution guaranteed